

MANYGAMES

Relatório Técnico Completo do Projeto

Sistema de Autenticação e Gestão de Utilizadores

Desenvolvido por: Ricardo Mor

Plataforma: GitHub Pages + Firebase (Firestore + Authentication)

Repositório: rikmor.github.io/many-games/

Data: Julho de 2026

ÍNDICE

1. Visão Geral do Projeto
2. Arquitetura e Tecnologias
3. Estrutura de Ficheiros
 4. firebase.js — Configuração Firebase
 5. shared.css — Estilos Globais
 6. nav-auth.js — Navegação Dinâmica
 7. login.html — Autenticação
 8. perfil.html — Página de Perfil
 9. definicoes.html — Definições do Utilizador
 10. privacidade.html — Política de Privacidade
 11. admin.html — Painel Administrativo
12. Base de Dados Firestore
13. Regras de Segurança Firestore
14. Firebase Authentication
15. Funcionalidades por Tipo de Conta
16. Fluxos de Utilizador
17. Páginas do Site (restantes)
18. Considerações de Segurança
19. Limitações e Melhorias Futuras

1. VISÃO GERAL DO PROJETO

O **ManyGames** é um site de empresa de jogos desenvolvido para apresentação escolar, alojado no GitHub Pages (hosting estático gratuito). O projeto inclui informação sobre dois jogos — **Kill of Reason** (MOBA) e **TGG** (jogo de cartas/tabuleiro) — loja, notícias, equipa, carreiras, redes sociais e um sistema completo de autenticação de utilizadores com Firebase.

No decorrer do desenvolvimento foram implementadas de raiz todas as funcionalidades de gestão de utilizadores: registo, login, contas de convidado temporárias, perfis públicos, página de definições, e um painel administrativo com gestão de utilizadores e outros dados do site.

Objetivos alcançados:

- Site completo com 19 páginas HTML independentes
- Sistema de autenticação seguro via Firebase Authentication (email/password + anónimo)
- Base de dados Firestore com coleções para utilizadores, produtos, notícias e formulários
- Painel de administração protegido com filtragem, exportação Excel e gestão de dados
- Página de perfil pública por utilizador com estatísticas e conquistas
- Definições de conta: avatar, banner, nickname, email, password, sessões e badges
- Contas de convidado com sessão temporária (apaga ao fechar o browser)

2. ARQUITETURA E TECNOLOGIAS

Tecnologias utilizadas:

Tecnologia	Versão / Detalhe	Função
HTML5 / CSS3 / JS	Vanilla (sem frameworks)	Frontend estático
Firebase Authentication	v10.12.0 (modular SDK)	Login, registo, sessões
Firebase Firestore	v10.12.0 (modular SDK)	Base de dados NoSQL
GitHub Pages	—	Hosting gratuito estático
ReportLab	Python	Geração deste relatório
DiceBear API	v8.x pixel-art	Avatares gerados automaticamente
SheetJS (xlsx)	CDN	Exportação Excel no admin

Modelo de arquitetura:

O site segue uma arquitetura **Jamstack** (JavaScript + APIs + Markup): todos os ficheiros são estáticos (HTML/CSS/JS) servidos pelo GitHub Pages, sem servidor próprio. As operações de autenticação e base de dados são tratadas pelo Firebase (Google Cloud) via chamadas diretas do browser usando o SDK modular ES6.

Os módulos Firebase são importados via CDN (www.gstatic.com/firebasejs) usando `type="module"` em scripts ES6. Para garantir que a validação de formulários funcione sempre (independentemente do carregamento dos módulos Firebase), a lógica de validação foi separada em scripts normais que expõem funções via `window._fb*`.

3. ESTRUTURA DE FICHEIROS

Ficheiro	Tipo	Descrição
firebase.js	JS Módulo	Configuração e exportações Firebase (Auth + Firestore)
nav-auth.js	JS Módulo	Dropdown de navegação dinâmico conforme estado de login
shared.css	CSS	Estilos globais partilhados por todas as páginas
shared.js	JS	Scripts partilhados: mobile nav, scroll reveal
login.html	HTML	Página de login/registo/convidado (3 tabs)
perfil.html	HTML	Página de perfil público do utilizador
definicoes.html	HTML	Configurações da conta do utilizador
privacidade.html	HTML	Política de Privacidade
admin.html	HTML	Painel administrativo (acesso restrito ao admin)
index.html	HTML	Página inicial
jogo.html	HTML	Página do jogo Kill of Reason
loja.html	HTML	Loja de produtos
noticias.html	HTML	Notícias e atualizações
equipa.html	HTML	Página da equipa
rh.html	HTML	Recrutamento / Carreiras
regulamento.html	HTML	Regulamento interno
sobre.html	HTML	Sobre a empresa
faq.html	HTML	Perguntas Frequentes
contacto.html	HTML	Formulário de contacto
instagram.html	HTML	Feed Instagram (dados do Firestore)
facebook.html	HTML	Página Facebook
reddit.html	HTML	Página Reddit
twitter.html	HTML	Página Twitter / X
videos.html	HTML	Vídeos do projeto

4. firebase.js — CONFIGURAÇÃO FIREBASE

Ficheiro central que inicializa o Firebase e exporta todas as instâncias e funções necessárias para os outros módulos. É importado por login.html, perfil.html, definicoes.html, nav-auth.js e admin.html.

Exportações:

- **auth** — instância de Firebase Authentication (getAuth)
- **db** — instância de Firestore (getFirestore)
- **doc, setDoc, updateDoc, deleteDoc** — operações de documento Firestore
- **collection, addDoc, getDocs, query, orderBy, onSnapshot** — operações de coleção

Configuração do projeto Firebase:

Project ID: manygames-2f8a7 | Auth Domain: manygames-2f8a7.firebaseio.com

5. shared.css — ESTILOS GLOBAIS

Ficheiro de estilos partilhado por todas as páginas do site. Define as variáveis CSS, tipografia, componentes de navegação e o dropdown do utilizador autenticado.

Variáveis CSS principais:

Variável	Valor	Uso
--accent	#e63946	Cor principal (vermelho)
--cyan	#00d4ff	Cor de destaque secundária
--bg	#0a0a12	Fundo da página
--card	#12121e	Fundo de cartões e inputs
--border	#1e1e30	Bordas e separadores
--text	#ffffff	Texto principal
--muted	#b0b0cc	Texto secundário / labels

Fontes utilizadas:

- **Bebas Neue** — títulos e botões (lettering em maiúsculas)
- **Rajdhani** — corpo de texto e navegação
- **Share Tech Mono** — labels, tags, elementos monoespaçados

Estilos adicionados para o sistema de autenticação:

- .nav-user-dd — dropdown do utilizador autenticado na navegação
- .nav-user-btn — botão com avatar + nickname na nav
- .nav-user-menu — menu dropdown com Perfil, Definições e Terminar Sessão

6. nav-auth.js — NAVEGAÇÃO DINÂMICA

Módulo ES6 partilhado por todas as páginas (exceto admin.html). Injeta dinamicamente o estado de autenticação na navegação sem necessidade de alterar o HTML de cada página.

Funcionamento:

- Importa onAuthStateChanged do Firebase Auth
- Observa o estado de autenticação em tempo real
- Se **não autenticado**: injeta o link simples 'Login' no #nav-user-area
- Se **autenticado (conta normal)**: injeta avatar + nickname + dropdown com Perfil, Definições e Logout
- Se **autenticado (convidado)**: injeta avatar + 'Convidado' + dropdown sem a opção Definições

Para contas normais, vai buscar ao Firestore:

- nickname — nome de exibição do utilizador
- avatarUrl — URL da imagem de perfil (ou gera automaticamente via DiceBear)

Inclusão nas páginas:

Todas as páginas incluem no final do <body>:

```
<script type="module" src="nav-auth.js"></script>
```

Placeholder HTML em todas as páginas:

```
<li id="nav-user-area"><a href="login.html">Login</a></li>
```

7. login.html — AUTENTICAÇÃO

Página principal de autenticação com três tabs: **Entrar**, **Registrar** e **Convidado**. Após autenticação bem-sucedida, redireciona automaticamente para perfil.html.

Arquitetura de scripts:

Para garantir robustez, a página usa **dois scripts separados**:

- **Script normal** — validação de formulários, lógica dos tabs (sem dependências Firebase). Chama funções `window._fbLogin`, `window._fbRegister`, `window._fbGuest`.
- **Script módulo** — importa Firebase e expõe as funções via `window.*`. O botão 'Criar Conta' começa desativado e ativa quando o módulo carrega.

Tab ENTRAR:

- Campos: Email + Palavra-passe
- Usa `browserLocalPersistence` — sessão mantida ao fechar/reabrir o browser
- Função: `signInWithEmailAndPassword(auth, email, password)`

Tab REGISTRAR:

- Campos: Nome completo, Nickname, Email, Palavra-passe, Confirmar Palavra-passe, Nacionalidade
- Checkboxes obrigatórias: confirmação de +13 anos e aceitação da Política de Privacidade
- Validações: formato do nickname (3-24 chars, letras/números/_/.), filtro de palavras impróprias (PT+EN), verificação de unicidade do nickname no Firestore
- Cria conta no Firebase Auth + documento na coleção `users` do Firestore
- Campos guardados: `name`, `nickname`, `nicknameLower`, `email`, `country`, `avatarUrl`, `stats`, `ageConfirmed`, `privacyAccepted`, `createdAt`

Tab CONVIDADO:

- Campos: Nickname, checkbox +13 anos, checkbox Política de Privacidade
- Usa `browserSessionPersistence` — sessão apaga ao fechar o browser
- Usa `signInAnonymously(auth)` — conta anónima no Firebase Auth
- Guarda documento no Firestore com flag `isGuest: true`
- Ao fechar o browser: apaga documento Firestore e conta Auth via REST API com `fetch` `keepalive:true`

Limpeza automática de convidados:

Quando o login.html carrega sem utilizador autenticado, verifica a coleção `users` e apaga automaticamente documentos de convidados com mais de 1 hora.

Erros traduzidos para português:

- auth/email-already-in-use, auth/invalid-email, auth/weak-password
- auth/user-not-found, auth/wrong-password, auth/invalid-credential
- auth/too-many-requests, auth/operation-not-allowed, auth/admin-restricted-operation

8. perfil.html — PÁGINA DE PERFIL

Página de perfil público do utilizador. Acesso restrito — redireciona para login.html se não autenticado. Página de visualização apenas (sem edição — edição feita em definicoes.html).

Componentes visuais:

- **Banner** — imagem de fundo personalizada (URL guardada no Firestore). Se não definido, usa gradiente decorativo com grelha vermelha.
- **Avatar** — imagem circular com borda vermelha. Se não definido, gera automaticamente via DiceBear API (pixel-art baseado no nickname).
- **Cabeçalho** — nickname (Bebas Neue), nome completo, país com emoji, data de membro.
- **Estatísticas** — grelha 4 colunas: Jogos, Vitórias (verde), Derrotas (vermelho), MVP (ciano).
- **Conquistas** — badges desbloqueadas e seleccionadas pelo utilizador nas Definições.

Sistema de conquistas (badges):

Badge	Condicao de desbloqueio
Primeiro Login	Sempre desbloqueado ao criar conta
Guerreiro KOR	stats.jogos >= 1
Veterano	stats.jogos >= 10
Primeira Vitoria	stats.vitorias >= 1
Em Forma	stats.vitorias >= 5
MVP	stats.mvp >= 1

Lógica de proteção:

- Se não autenticado: mostra div 'ACESSO RESTRITO' com botão Fazer Login
- Se autenticado mas documento Firestore não existe: faz signOut e redireciona para login
- Se convidado: mostra banner de aviso 'Conta Temporária' + limpeza ao fechar browser

Limpeza de convidado ao fechar browser:

Ao detetar utilizador anónimo, guarda o ID token do Firebase e renova a cada 55 minutos. No evento pagehide, usa fetch com keepalive: true para chamar a API REST do Firestore (DELETE) e a API REST do Firebase Auth (accounts:delete), garantindo a execução mesmo após o browser fechar.

9. definicoes.html — DEFINIÇÕES DO UTILIZADOR

Página de configurações da conta. Acesso restrito — redireciona para login se não autenticado, e para perfil se for conta de convidado (convidados têm acesso limitado).

Secções disponíveis:

Seccao	Funcao	Campo Firestore
Avatar	URL da imagem de perfil (200x200 px)	avatarUrl
Banner	URL do banner do perfil (1200x300 px)	bannerUrl
Nickname	Alterar nickname (formato + unicidade)	nickname, nicknameLower
Email	Alterar email (requer re-autenticacao)	email (Auth + Firestore)
Palavra-passe	Alterar palavra-passe (requer pass atual)	Firebase Auth
Terminar Sessoes	Encerra sessao noutros dispositivos	Firebase Auth (revoga tokens)
Badges visiveis	Escolher conquistas a mostrar no perfil	visibleBadges (array)

Notas técnicas:

- Avatar e Banner: campo de URL em vez de upload de ficheiro (compatível com hosting estático)
- Re-autenticação: operações sensíveis (email, password, sessões) usam `reauthenticateWithCredential` com `EmailAuthProvider.credential`
- Terminar sessões noutros dispositivos: chama `updatePassword` com a mesma palavra-passe — isto invalida todos os refresh tokens no Firebase
- Badges visíveis: guardadas como array de IDs em `users/{uid}.visibleBadges`
- Convidados: são automaticamente redirecionados para `perfil.html` ao acederem às definições

10. privacidade.html — POLÍTICA DE PRIVACIDADE

Página de Política de Privacidade criada de raiz conforme os requisitos legais mínimos para recolha de dados de utilizadores. Acessível no dropdown 'Suporte' da navegação e linkada no formulário de registo.

Conteúdo:

- Dados recolhidos: nome, nickname, email, password (hash Firebase), nacionalidade, confirmação de idade
- Idade mínima: 13 anos (COPPA/GDPR básico)
- Uso dos dados: gestão de conta, login, comunicações
- Armazenamento: Firebase (Google Cloud) com encriptação de passwords
- Direitos do utilizador: acesso, correção e eliminação via contacto

11. admin.html — PAINEL ADMINISTRATIVO

Painel de administração completo com autenticação exclusiva para o email `adm@manygames.com`. Qualquer outro utilizador autenticado é automaticamente desconectado ao tentar aceder. O link de acesso está discretamente no logo do footer da página `login.html`.

Secções do painel:

Secção	Coleção Firestore	Funcionalidades
Dashboard	Todas	Contadores em tempo real, resumo geral
Encomendas	encomendas	Ver, filtrar, exportar Excel, eliminar
Candidaturas	candidaturas	Ver CVs, filtrar, exportar, eliminar
Contactos	contactos	Ver mensagens, filtrar, exportar, eliminar
Newsletter	newsletter	Gerir subscritores, exportar, eliminar
Produtos	produtos	Criar, editar, eliminar produtos da loja
Vídeos	videos	Gerir vídeos do site
Instagram	instagram	Gerir posts do feed Instagram
Facebook	facebook	Gerir posts Facebook
Reddit	reddit	Gerir posts Reddit
Twitter / X	twitter	Gerir posts Twitter
Notícias	noticias	Criar e gerir artigos de notícias
Utilizadores	users	Ver perfis, filtrar por continente/data, modal de detalhes

Funcionalidades globais do admin:

- Atualização em tempo real via `onSnapshot` do Firestore (sem F5)
- Badge de contagem animado em cada secção da sidebar
- Pesquisa por texto em todas as coleções
- Filtro por data em todas as coleções
- Filtro por continente na secção Utilizadores (África, América do Norte, América do Sul, Ásia, Europa, Oceânia)
- Seleção múltipla com checkboxes para ações em lote
- Modal de detalhe para ver informação completa de cada registo
- Exportação para Excel (XLSX) via SheetJS — exporta tudo ou só selecionados
- Botão 'Limpar tudo' com confirmação para cada coleção

Limpeza automática de convidados:

Quando a secção Utilizadores carrega, verifica automaticamente todos os documentos com isGuest: true e apaga os que têm mais de 1 hora (campo createdAt).

12. BASE DE DADOS FIRESTORE

Estrutura da coleção users (documento por utilizador):

Campo	Tipo	Descrição
name	string	Nome completo do utilizador
nickname	string	Nickname público (case-sensitive)
nicknameLower	string	Nickname em minúsculas (para verificação de unicidade)
email	string	Endereço de email
country	string	Código de país ISO 3166-1 alpha-2 (ex: PT, BR)
avatarUrl	string	URL da imagem de avatar
bannerUrl	string	URL da imagem de banner do perfil
isGuest	boolean	true se conta de convidado anónimo
ageConfirmed	boolean	true se confirmou ter 13+ anos
privacyAccepted	boolean	true se aceitou a Política de Privacidade
stats.jogos	number	Número total de jogos disputados
stats.vitorias	number	Número de vitórias
stats.derrotas	number	Número de derrotas
stats.mvp	number	Número de vezes eleito MVP
visibleBadges	array	IDs das conquistas seleccionadas para mostrar no perfil
createdAt	string	Data/hora de criação (ISO 8601)

Outras coleções do Firestore:

Coleção	Campos principais	Acesso
produtos	nome, preco, imagem, descricao, categoria	Leitura pública, escrita só admin
noticias	titulo, data, conteudo, imagem, autor	Leitura pública, escrita só admin
videos	titulo, url, thumbnail, descricao	Leitura pública, escrita só admin
instagram	caption, img, likes, url, timestamp	Leitura pública, escrita só admin
facebook	caption, img, likes, url, timestamp	Leitura pública, escrita só admin
reddit	titulo, url, upvotes, timestamp	Leitura pública, escrita só admin
twitter	texto, likes, retweets, timestamp	Leitura pública, escrita só admin
vagas	titulo, area, tipo, descricao	Leitura pública, escrita só admin

encomendas	nome, email, produto, quantidade, data	Criação pública, leitura só admin
candidaturas	nome, email, vaga, motivacao, linkedin	Criação pública, leitura só admin
contactos	nome, email, assunto, mensagem, data	Criação pública, leitura só admin
newsletter	email, data	Criação pública, leitura só admin

13. REGRAS DE SEGURANÇA FIRESTORE

As regras de segurança do Firestore controlam quem pode ler e escrever em cada coleção. São definidas na Firebase Console e avaliadas no servidor — não podem ser contornadas pelo cliente.

Resumo das regras:

Colecao	Leitura	Escrita
produtos, noticias, videos, instagram, facebook, reddit, twitter, vagas	Qualquer pessoa	Utilizador autenticado (qualquer)
encomendas, candidaturas, contactos, newsletter	So admin autenticado	Qualquer pessoa (create only)
users/{userId}	Qualquer autenticado	Proprio utilizador OU admin (adm@manygames.com)
/* (resto)	Negado	Negado

Regra especial da coleção users:

A coleção users permite escrita se `request.auth.uid == userId` (utilizador a editar o seu próprio perfil) **OU** se `request.auth.token.email == 'adm@manygames.com'` (administrador). Isto permite ao admin gerir documentos de qualquer utilizador.

14. FIREBASE AUTHENTICATION

Métodos de autenticação ativos:

- **Email/Password** — login e registo com email e palavra-passe encriptada (hash bcrypt pelo Firebase)
- **Anónimo** — contas de convidado sem credenciais

Persistência de sessão:

Tipo de conta	Persistência	Comportamento
Conta normal (email)	browserLocalPersistence	Mantém sessão ao fechar/reabrir o browser
Conta convidado	browserSessionPersistence	Sessão termina ao fechar a aba/browser

Segurança das palavras-passe:

As palavras-passe são geridas exclusivamente pelo Firebase Authentication. O cliente **nunca tem acesso** à password em texto simples — é enviada por HTTPS e armazenada como hash (bcrypt) nos servidores da Google. O projeto não guarda passwords no Firestore.

Proteção do admin:

O admin.html verifica no `onAuthStateChanged` se o email do utilizador é `adm@manygames.com`. Se qualquer outro utilizador tentar aceder, é imediatamente desconectado e vê a mensagem "Acesso negado".

15. FUNCIONALIDADES POR TIPO DE CONTA

Funcionalidade	Conta Normal	Convidado	Admin
Ver perfil	Sim	Sim	Sim
Definições de conta	Sim	Nao	Sim
Mudar avatar / banner	Sim	Nao	Sim
Mudar nickname	Sim	Nao	Sim
Mudar email	Sim	Nao	Sim
Mudar palavra-passe	Sim	Nao	Sim
Terminar sessão noutros dispositivos	Sim	Nao	Sim
Escolher badges visíveis	Sim	Nao	Sim
Dropdown nav com Definições	Sim	Nao	Sim
Sessão persistente	Sim	Nao	Sim
Acesso ao admin	Nao	Nao	Sim
Eliminar dados no admin	Nao	Nao	Sim
Auto-eliminação ao fechar browser	Nao	Sim	Nao

16. FLUXOS DE UTILIZADOR

Fluxo de Registo:

- 1. Utilizador acede a login.html e clica na tab 'Registar'
- 2. Preenche: nome, nickname, email, palavra-passe (×2), nacionalidade
- 3. Confirma checkboxes: +13 anos e Política de Privacidade
- 4. Validação client-side: formato nickname, palavras impróprias, passwords iguais
- 5. Verificação no Firestore: nickname único (case-insensitive via nicknameLower)
- 6. Firebase cria conta em Authentication com email+password encriptada
- 7. updateProfile guarda o nickname como displayName no Auth
- 8. setDoc cria documento na coleção users com todos os campos
- 9. Redireciona para perfil.html

Fluxo de Login:

- 1. Utilizador acede a login.html, tab 'Entrar'
- 2. Preenche email e palavra-passe
- 3. signInWithEmailAndPassword com browserLocalPersistence
- 4. Redireciona para perfil.html
- 5. nav-auth.js deteta o utilizador e injeta avatar+nickname no dropdown da nav

Fluxo de Convidado:

- 1. Utilizador acede a login.html, tab 'Convidado'
- 2. Preenche nickname + confirmações
- 3. setPersistence(browserSessionPersistence) + signInAnonymously
- 4. Cria documento Firestore com isGuest:true
- 5. Regista listener pagehide com fetch keepalive para limpeza
- 6. Redireciona para perfil.html (mostra banner de conta temporária)
- 7. Ao fechar browser: apaga documento Firestore + conta Auth via REST API

Fluxo de Acesso ao Admin:

- 1. Utilizador acede a login.html e clica no logo MANYGAMES no footer
- 2. admin.html carrega e verifica onAuthStateChanged
- 3. Se email == adm@manygames.com: mostra painel
- 4. Se outro utilizador autenticado: signOut imediato + mensagem de acesso negado
- 5. Se não autenticado: mostra ecrã de login do admin

17. PÁGINAS DO SITE (RESTANTES)

Todas as páginas do site partilham a mesma navegação com dropdowns, estilos via shared.css e o módulo nav-auth.js para o estado de autenticação.

Página	Conteúdo	Dados Firestore
index.html	Página inicial com hero, jogos e destaques	produtos, noticias
jogo.html	Página do Kill of Reason (MOBA)	—
loja.html	Loja com produtos, carrinho e formulário de encomenda	produtos, encomendas
noticias.html	Feed de notícias e artigos	noticias
equipa.html	Apresentação da equipa ManyGames	—
rh.html	Vagas e formulário de candidatura	vagas, candidaturas
regulamento.html	Regulamento interno da empresa	—
sobre.html	Sobre a empresa, missão e valores	—
faq.html	Perguntas frequentes com accordion	—
contacto.html	Formulário de contacto e suporte	contactos
instagram.html	Feed de posts Instagram	instagram
facebook.html	Posts Facebook	facebook
reddit.html	Posts Reddit	reddit
twitter.html	Posts Twitter / X	twitter
videos.html	Galeria de vídeos	videos

Dropdowns da navegação:

- **Recursos Humanos:** Equipa, Carreiras, Regulamento
- **Suporte:** Sobre, FAQ, Contacto, Política de Privacidade
- **Redes Sociais:** Instagram, Facebook, Reddit, Twitter/X, Vídeos
- **Utilizador (dinâmico):** Login / [Avatar + Nickname > Perfil, Definições, Logout]

18. CONSIDERAÇÕES DE SEGURANÇA

Medidas implementadas:

- **Passwords encriptadas** — Firebase Authentication usa bcrypt. Nunca são guardadas em texto simples.
- **Regras Firestore** — cada coleção tem regras específicas de leitura/escrita avaliadas no servidor.
- **Proteção do admin** — verificação do email no `onAuthStateChanged` + logout automático de utilizadores não autorizados.
- **Filtro de nicknames** — lista de palavras impróprias PT+EN com normalização de acentos e substituições comuns.
- **Validação de idade** — checkbox obrigatória de confirmação de 13+ anos no registo.
- **HTTPS obrigatório** — GitHub Pages serve exclusivamente via HTTPS; Firebase rejeita requests HTTP.
- **Persistência por tipo** — contas normais usam `localStorage` (persistente), convidados usam `sessionStorage` (temporário).
- **Re-autenticação** — operações sensíveis (mudar email/password) exigem confirmação da password atual.

Limitações conhecidas:

- API Key Firebase visível no código client-side — é comportamento normal e esperado pelo Firebase; as regras Firestore fazem a proteção real.
- Contas anónimas no Firebase Auth não são eliminadas automaticamente no plano gratuito (Spark) — acumulam-se mas são inativas.
- Sem rate limiting próprio — depende dos limites nativos do Firebase (100 registos/hora por IP).

19. LIMITAÇÕES E MELHORIAS FUTURAS

Limitações atuais:

- Hosting estático — sem lógica server-side, sem Cloud Functions (plano pago Blaze necessário)
- Sem envio de emails — verificação de email, recuperação de password dependem do Firebase nativo
- Estatísticas manuais — jogos, vitórias e MVP têm de ser atualizados manualmente pelo admin
- Sem chat ou funcionalidades em tempo real entre utilizadores
- Sem sistema de amigos ou comunidade

Melhorias possíveis:

- Implementar Cloud Functions (Blaze) para limpeza automática de contas anónimas e envio de emails personalizados
- Adicionar login com Google (OAuth) — já recomendado pela Firebase Console
- Sistema de leaderboard com ranking público de jogadores
- Perfis públicos acessíveis por URL (ex: /perfil?user=ShadowBlade99)
- Sistema de conquistas ligado a dados reais do jogo Kill of Reason
- Notificações in-app via Firestore onSnapshot
- Modo escuro / claro toggle (o site já usa variáveis CSS que facilitam isso)

Relatório gerado automaticamente em Julho de 2026

ManyGames · Porto, Portugal · rikmor.github.io/many-games/